

ACADEMIC PROJECT REPORT

Department of Computer Science & Engineering

HUMAN RESOURCE MANAGEMENT SYSTEM (HRMS)

A Robust Core Java Application for Workforce Administration, Payroll
Analytics & File Persistence

Core Java • OOP Design • Apache Maven • JUnit 5

SUBMITTED BY:

Student Name: Priyanshu Choudhary

Program: B.Tech (Computer Science & Engg.)

Course: Java Programming Laboratory

Language: Java Standard Edition (JDK 26)

Build Tool: Apache Maven 3.9

PROJECT METADATA:

Project Type: Coursework Mini-Project

Domain: Enterprise Software / HRMS

Architecture: 3-Tier Layered Architecture

Storage: Flat-File CSV (Buffered I/O)

Academic Year: 2026–2027

Certificate of Completion

This is to certify that the project entitled "**Human Resource Management System (HRMS)**" submitted by **Priyanshu Choudhary** for the fulfillment of the requirements for the award of Bachelor of Technology in Computer Science & Engineering is an authentic record of original project work carried out under proper faculty guidance.

The system demonstrates the practical application of Core Java paradigms, Object-Oriented Software Engineering principles, automated unit testing with JUnit 5, and file input/output streams. The source code and documentation have been thoroughly reviewed and validated.

Faculty Mentor / Guide

Head of Department (CSE)

Student Declaration

I, **Priyanshu Choudhary**, declare that the project titled "**Human Resource Management System (HRMS)**" has been independently conceptualized, coded, and tested by me.

All algorithmic designs, data serialization routines, business calculations, and unit tests presented in this report represent bona fide work. External reference materials and standard library frameworks utilized during development have been appropriately cited.

Priyanshu Choudhary

Department of Computer Science & Engineering

Executive Summary & Abstract

The **Human Resource Management System (HRMS)** is a lightweight, high-performance console utility engineered to handle fundamental workforce operations without the deployment overhead of external database servers or complex graphical interfaces. Built entirely with **Core Java**, it emphasizes clean Object-Oriented design, safe state encapsulation, and crash-resilient input validation.

The system features a structured 3-tier architecture: the *Model layer* (encapsulating employee entity data), the *Service layer* (governing business operations, duplicate prevention, salary analytics, and CSV persistence), and the *Presentation layer* (an interactive CLI terminal menu). File durability is guaranteed through standard buffered character streams writing to `data/employee.txt`. System correctness is validated via 11 automated unit tests in **JUnit 5** with a 100% pass rate.

100% TEST PASS RATE	11 JUNIT 5 TESTS	10 CLI OPERATIONS	0 EXTERNAL DBS NEEDED
---	--	---	---

Table of Contents

1.	Introduction, Motivation & Problem Statement	Page 4
2.	System Analysis & Requirements Specification	Page 5
3.	System Architecture & Design Principles	Page 6
4.	Detailed Component Implementation	Page 7
5.	Data Persistence & Storage Architecture	Page 8
6.	Testing Methodology & Quality Assurance	Page 9
7.	Execution Output & Interface Demonstrations	Page 10
8.	Conclusion, Future Scope & References	Page 11

1. Introduction

1.1 Background & Motivation

Human Resource departments across enterprises require reliable software tools to catalog employee rosters, verify unique identities, update compensation structures, and generate payroll metrics. While enterprise ERP systems (e.g., SAP, Workday) cater to large organizations, they carry steep licensing costs, substantial memory footprints, and heavy database prerequisites.

For academic study, internal tooling, and small-scale operations, a self-contained, console-driven Java application offers an ideal showcase of fundamental software engineering principles: **Object-Oriented Programming (OOP)**, **Defensive Input Validation**, and **Reliable File I/O**.

1.2 Problem Statement

The goal was to design, construct, and verify a complete, standalone Human Resource Management System in Java capable of performing all essential workforce administration tasks without relying on external UI frameworks or third-party databases.

The application must provide intuitive terminal interaction, prevent runtime exceptions caused by invalid user inputs, maintain data durability across sessions via flat-file persistence, and provide business analytics including gross payroll and salary averages.

1.3 Objectives & Scope

Core Engineering Objectives

Deliver an academically sound, production-grade Core Java application following clean architecture, verifiable through automated JUnit test suites.

- **Robust Data Modeling:** Encapsulate employee attributes (ID, Name, Salary) with proper getters, setters, and formatters.
- **Data Integrity & Validation:** Enforce unique ID constraints, reject negative salaries, and validate strings.
- **Crash-Resilient CLI:** Guard terminal inputs against `InputMismatchException` and scanner newline bugs.
- **Durable Storage:** Serialize employee lists to comma-separated text files (`data/employee.txt`) with automatic directory creation and error recovery.
- **Payroll Intelligence:** Compute total payroll, average salary, highest paid, and lowest paid employees in real time.

2. System Analysis & Requirements

2.1 Functional Requirements

Req ID	Feature	Description & Validation Rules
FR-01	Add Employee	Registers employee with ID, Name, and Salary. Rejects duplicate IDs and negative compensation figures.
FR-02	Display Directory	Renders formatted ASCII table of all staff with aligned ID, Name, and INR currency formatting.
FR-03	Search by ID	Executes fast exact-match lookup on employee ID; returns full record or an informative not-found message.
FR-04	Search by Name	Case-insensitive substring search matching partial or full employee names across the roster.
FR-05	Update Record	Allows selective modification of Name, Salary, or both with immediate validation checks.
FR-06	Remove Employee	Deletes employee by ID with mandatory user confirmation prompt (<code>y/N</code>).
FR-07	HR Statistics	Computes total workforce count, gross payroll, average salary, and top/lowest earners.
FR-08	File Persistence	Loads records on launch; supports on-demand saving and automatic save prompt upon application exit.

2.2 Non-Functional Requirements

- **Fault Tolerance:** Entering non-numeric text at integer/decimal prompts re-prompts the user gracefully.
- **Portability:** Compiles to standard Java bytecode runnable across macOS, Linux, and Windows.
- **Efficiency:** All lookup and aggregation operations execute in in-memory time complexity: $\mathcal{O}(n)$.
- **Maintainability:** Clean decoupling of concerns across `model`, `service`, and `app` packages.

2.3 Environmental Specifications

Component	Specification
Language & Runtime	Java Standard Edition (JDK 17 / 21 / 26 compatible)
Build Tool	Apache Maven 3.8+ (exec-maven-plugin 3.5.0, compiler 3.15.0)
Testing Framework	JUnit Jupiter 5.13.4
Persistence Medium	Structured Delimited File (<code>data/employee.txt</code>)

3. System Architecture & Design

3.1 Three-Tier Layered Architecture

The application implements a decoupled, three-tier architectural pattern. This ensures high cohesion within classes and low coupling between layers:

```

+-----+
|                PRESENTATION LAYER (app.Main)                |
| - Interactive Menu Controller   - Crash-Proof Input Parsers   - ASCII Renderers |
+-----+
|          |
|          v          |
+-----+
|                SERVICE / BUSINESS LAYER                    |
| service.EmployeeService      | service.FileService          |
| - In-Memory List Management (ArrayList) | - BufferedReader / BufferedWriter |
| - Duplicate Validation & Search Logic   | - CSV Serialization & Parsing         |
| - Real-time Payroll Statistical Engine   | - Directory Creation & Auto-Recovery |
+-----+
|          |
|          v          |
+-----+
|                DOMAIN / MODEL LAYER                        |
|                model.Employee                             |
| - Private State: id, name, salary      | - CSV Helpers: toCsvString, fromCsv    |
| - Accessors / Mutators (Encapsulation) | - Identity: equals() / hashCode()    |
+-----+
|          |
|          v          |
+-----+
|                PERSISTENCE STORAGE                         |
|                data/employee.txt (CSV Format)              |
+-----+

```

3.2 Object-Oriented Principles Applied

1. **Encapsulation:** All fields in `Employee` are declared `private`. State modifications occur only through validated methods (`setSalary`, `setName`).
2. **Single Responsibility Principle (SRP):** `Employee` maintains entity data, `EmployeeService` executes business calculations, `FileService` isolates disk I/O, and `Main` orchestrates user dialogue.
3. **Defensive Copying:** `EmployeeService.getAllEmployees()` returns an unmodifiable list copy (`Collections.unmodifiableList`), safeguarding internal lists from external mutation.
4. **Separation of Concerns:** File persistence is parameterized, allowing unit tests to inject temporary directories without mutating production records.

4. Implementation Details

4.1 Model Layer: `Employee.java`

The domain model encapsulates employee state and provides bidirectional CSV mapping:

```
package model;
import java.util.Objects;

public class Employee {
    private int id;
    private String name;
    private double salary;

    public Employee(String name, int id, double salary) {
        this.name = name; this.id = id; this.salary = salary;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }

    public String toCsvString() {
        return id + "," + name + "," + String.format(java.util.Locale.US, "%.2f", salary);
    }
    public static Employee fromCsvString(String line) {
        if (line == null || line.trim().isEmpty() || line.startsWith("#")) return null;
        String[] parts = line.trim().split(",", -1);
        if (parts.length < 3) return null;
        try {
            return new Employee(parts[1].trim(), Integer.parseInt(parts[0].trim()),
                                Double.parseDouble(parts[2].trim()));
        } catch (NumberFormatException e) { return null; }
    }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return id == ((Employee) o).id;
    }
    @Override public int hashCode() { return Objects.hash(id); }
}
```

4.2 Business Service Layer: `EmployeeService.java`

Governs workforce collections, integrity checks, and analytics:

- `addEmployee(Employee e)` : Validates unique ID before appending to internal list.
- `findEmployeesByName(String q)` : Performs case-insensitive substring search.
- `getTotalPayroll()` & `getAverageSalary()` : Computes statistical metrics dynamically.
- `printEmployeeTable(List)` : Formats employee rows into an ASCII tabular grid.

5. Data Persistence & Storage Architecture

5.1 Flat-File Storage Architecture

The persistence subsystem is implemented in `FileService.java`. The system utilizes Comma-Separated Values (CSV) instead of Java binary object serialization. This delivers human readability, interoperability with external spreadsheet tools, and zero vulnerability to `InvalidClassException` during class evolution.

File Location & Schema

Persistent storage path: `HRMS/data/employee.txt`. Directory is dynamically created upon first run.

5.2 CSV Data Specification

Lines starting with `#` denote comment or header metadata. Data rows follow the strict format:

`ID,Name,Salary:`

```
# ID,Name,Salary
1,Snehil,100000.00
2,Dhyey,200000.00
3,Kartikey,110000.00
4,Kislay,150000.00
```

5.3 Persistence Mechanics in `FileService.java`

```
public boolean saveEmployees(List employees) {
    File file = new File(filePath);
    File parentDir = file.getParentFile();
    if (parentDir != null && !parentDir.exists()) parentDir.mkdirs();

    try (BufferedWriter writer = new BufferedWriter(new FileWriter(file))) {
        writer.write("# ID,Name,Salary");
        writer.newLine();
        for (Employee e : employees) {
            writer.write(e.toCsvString());
            writer.newLine();
        }
        return true;
    } catch (IOException e) { return false; }
}
```

5.4 Fault Tolerance & Recovery Features

- **Automatic Directory Creation:** Invokes `parentDir.mkdirs()` if `data/` is absent.
- **Resource Safety:** Uses Java try-with-resources to guarantee closing stream descriptors.
- **Corrupt Line Resilience:** Skips empty, malformed, or non-numeric records gracefully.

6. Testing & Quality Assurance

6.1 Testing Strategy

The verification suite uses **JUnit 5 (Jupiter)** via Maven Surefire. Tests are isolated using `@BeforeEach` and temporary directory injections (`@TempDir`) to test disk I/O without modifying application records.

6.2 Automated Test Execution Matrix

Test Class	Test Method	Target Assertion	Result
EmployeeServiceTest	<code>testAddEmployeeSuccess</code>	Verifies employee insertion & count	PASS
	<code>testAddDuplicateEmployee</code>	Rejects duplicate IDs	PASS
	<code>testFindEmployeeById</code>	Exact ID match & null on absent	PASS
	<code>testFindEmployeesByName</code>	Case-insensitive substring search	PASS
	<code>testUpdateSalary</code>	Salary update & negative check	PASS
	<code>testUpdateName</code>	Name update & blank rejection	PASS
	<code>testRemoveEmployee</code>	Deletion by ID and post-state	PASS
	<code>testPayrollStatistics</code>	Payroll, averages, top & lowest	PASS
FileServiceTest	<code>testSaveAndLoadEmployees</code>	Roundtrip file persistence (<code>@TempDir</code>)	PASS
	<code>testLoadNonExistentFile</code>	Returns empty list if file absent	PASS
	<code>testCsvParsing</code>	Header and corrupt line filtering	PASS

6.3 Maven Surefire Test Execution Output

```
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running service.FileServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.220 s
[INFO] Running service.EmployeeServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.094 s
[INFO]
[INFO] Results: Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS (Total time: 6.315 s)
```

7. Program Output & Demonstrations

7.1 Main Menu & Directory View

```
=====
                HUMAN RESOURCE MANAGEMENT SYSTEM (HRMS)
=====
Successfully loaded 4 employee record(s) from data/employee.txt.

----- MAIN MENU -----
[1] Add New Employee           [6] Remove Employee
[2] View All Employees         [7] View HR & Payroll Statistics
[3] Search Employee by ID     [8] Save Records to File
[4] Search Employee by Name   [9] Reload Records from File
[5] Update Employee (Salary) [0] Exit Application

Enter your choice (0-9): 2

--- [2] View All Employees ---
+-----+-----+-----+-----+
| ID    | Name                | Salary (INR) |
+-----+-----+-----+-----+
| 1     | Snehil              | 100000.00    |
| 2     | Dhyey               | 200000.00    |
| 3     | Kartikey            | 110000.00    |
| 4     | Kislay              | 150000.00    |
+-----+-----+-----+-----+
Total Employees: 4
```

7.2 Search & HR Analytics Output

```
Enter your choice (0-9): 4
--- [4] Search Employee by Name ---
Enter name or partial name: kar
Found 1 matching employee(s):
+-----+-----+-----+-----+
| ID    | Name                | Salary (INR) |
+-----+-----+-----+-----+
| 3     | Kartikey            | 110000.00    |
+-----+-----+-----+-----+

Enter your choice (0-9): 7
--- [7] HR & Payroll Statistics ---
+-----+-----+-----+-----+
| Metric                | Value                |
+-----+-----+-----+-----+
| Total Employees       | 4                    |
| Total Monthly Payroll | INR 560000.00        |
| Average Salary        | INR 140000.00        |
| Highest Paid Employee | Dhyey (INR 200000.00) |
| Lowest Paid Employee  | Snehil (INR 100000.00) |
+-----+-----+-----+-----+
```

8. Conclusion & Future Scope

8.1 Project Conclusion

The **Human Resource Management System (HRMS)** successfully demonstrates how core Java technologies and disciplined software engineering practices can deliver a robust, functional workforce utility without unnecessary graphical baggage or external database engines.

Key engineering outcomes achieved:

- **Strict Encapsulation:** Complete separation between domain entity representation and service logic.
- **Defensive Programming:** Comprehensive input validation safeguarding against runtime scanner failures.
- **Persistence:** Human-readable, fault-tolerant CSV file persistence supporting automatic loading and saving.
- **Empirical Verification:** 11 automated JUnit 5 tests confirming 100% functional correctness.

8.2 Future Scope & Enhancements

1. **Relational DB Integration:** Upgrading from CSV flat files to SQLite or PostgreSQL via JDBC / JPA.
2. **Authentication & RBAC:** Adding administrative logins and role-based operational permissions.
3. **Web RESTful Services:** Wrapping the service layer with Spring Boot to expose REST APIs for mobile/web apps.
4. **Departmental Organization:** Extending the `Employee` entity with department hierarchies and leave tracking.

9. References & Bibliography

- Bloch, Joshua. *Effective Java* (3rd Edition). Addison-Wesley Professional.
- Eckel, Bruce. *Thinking in Java* (4th Edition). Prentice Hall.
- Oracle Corporation. *Java Platform, Standard Edition Documentation (JDK 26)*.
- JUnit 5 Team. *JUnit 5 User Guide*. <https://junit.org/junit5/docs/current/user-guide/>
- Apache Software Foundation. *Apache Maven Project Documentation*. <https://maven.apache.org/>